FPGA-Based Adaptive Beamformer for 5G NR

Project Update Report — Phase 1 Complete

Date: April 25, 2026 — Phases 1.1–1.5 Complete — Phase 2 Starting

Executive Summary

This report documents the completion of all five sub-phases of Phase 1 of the FPGA-Based Adaptive Beamformer project. The project implements a beamforming receiver for a 5G New Radio (NR) base station on an FPGA, targeting an 8-element Uniform Linear Array (ULA) with a conventional delay-and-sum beamformer in Phase 1 and an adaptive LMS beamformer in Phase 2.

Phase 1 is fully complete: the floating-point golden reference and fixed-point analysis were verified in MATLAB, all five SystemVerilog modules were written, functional co-simulation achieved 512/512 sample match, and Vivado synthesis confirmed correct DSP48 inference at 17.78% utilisation on an Artix-7 `xc7a35tcpg236-1`. The design occupies under 4% of available LUTs, leaving ample headroom for the Phase 2 LMS engine.

Project Plan

Phase 1 — Non-Adaptive Beamformer

1.1. Algorithm simulation (MATLAB, floating-point) — Golden reference. Delay-and-sum beamformer, analytical validation of beam direction, array gain, and SINR improvement. **[Complete]**

1.2. Fixed-point analysis (MATLAB) — Word-length determination for every pipeline stage. SINR degradation and pattern deviation verified. Test vectors exported. **[Complete]**

1.3. RTL implementation (SystemVerilog) — Five modules: pipelined complex multiplier, complex accumulator, weight ROM, top-level wrapper, testbench. **[Complete]**

1.4. Co-simulation and verification — 512/512 output samples match expected_output.hex to within ±1 LSB. Directed all-zero test also passed. **[Complete]**

1.5. Synthesis (Vivado) — Synthesised for Artix-7 `xc7a35tcpg236-1`. All 16 DSP48E1 blocks correctly inferred. LUT utilisation 3.90%. **[Complete]**

Phase 2 — Adaptive LMS Beamformer

2.1. LMS algorithm simulation (MATLAB) — Validate LMS weight-update convergence from an arbitrary start using only a pilot reference signal. **[Next]**

2.2. Fixed-point LMS analysis (MATLAB) — Verify convergence with finite-precision arithmetic. Determine word lengths for the weight accumulator, error signal, step size.

2.3. HLS implementation (Vitis HLS) — Describe the LMS update in C++ with `ap_fixed` types. Co-simulate against MATLAB reference, export packaged RTL.

2.4. Integration and verification — Replace Phase 1 weight ROM with writeable register bank. Add HLS LMS engine and pilot-sequencer FSM. Verify weight convergence.

2.5. Synthesis and comparison — Synthesise full adaptive system. Document additional resource cost and power delta vs Phase 1.

System Parameters

Parameter	Value
Array type	Uniform Linear Array (ULA)
Number of elements M	8
Element spacing	$d/\lambda = 0.5$ (half-wavelength)
Target signal angle	30° from broadside
Interferer angle	−20° from broadside
Input SNR	10 dB
Input SIR	0 dB (equal power)
Time samples	512
Random seed	42 (fixed for reproducibility)
Target FPGA	Artix-7 `xc7a35tcpg236-1`
Target clock	200 MHz

Table 1: Simulation and implementation parameters.

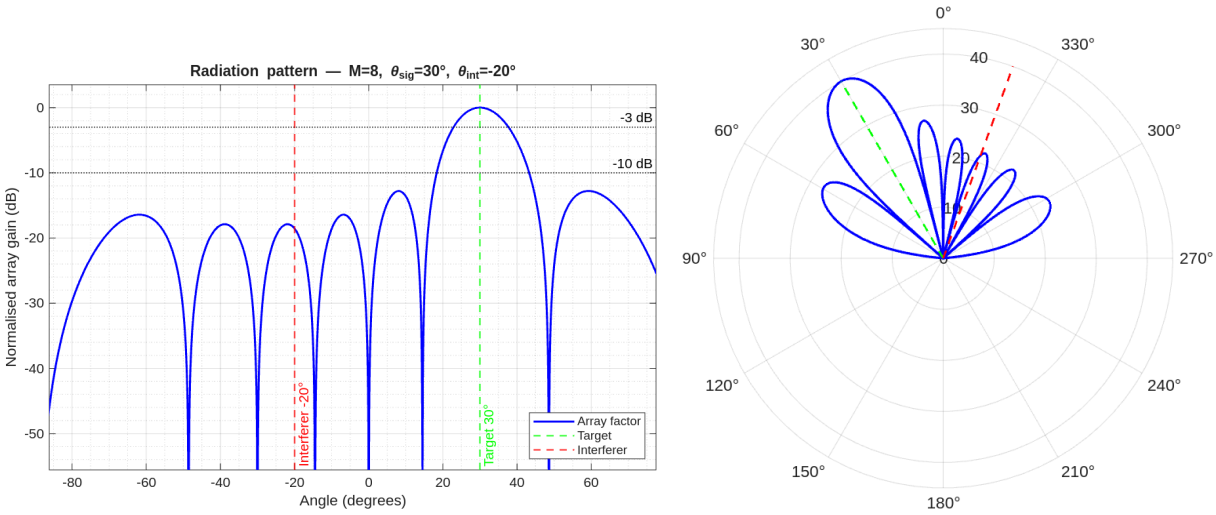
Phase 1.1 — Floating-Point Simulation

The conventional delay-and-sum beamformer was implemented in MATLAB. Weights are the normalised steering vector toward the target:

$$\mathbf{w} = \frac{\mathbf{a}(\theta_{\text{sig}})}{\|\mathbf{a}(\theta_{\text{sig}})\|}, \qquad y(n) = \mathbf{w}^H \mathbf{x}(n)$$

Metric	Value	Notes
Array gain (signal)	+9.03 dB	$\approx 10\log_{10}(8)$ ✓
Array gain (interferer)	−9.53 dB	Suppression at −20°
Input SINR	−2.55 dB	Before beamforming
Output SINR	+15.78 dB	After beamforming
SINR improvement	18.33 dB	Golden reference

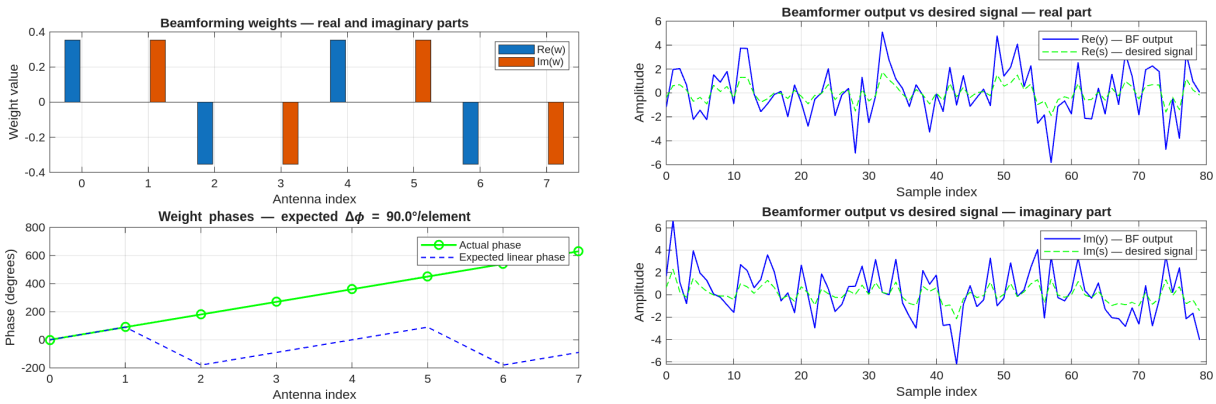
Table 2: Phase 1.1 verified metrics.



(a) Radiation pattern. Main lobe at 30°, null near the interferer at −20°.

(b) Polar radiation pattern.

Figure 1: Phase 1.1 — Array radiation patterns.



(a) Weight phases. Linear 90°/element progression confirms correct steering.

(b) Beamformer output vs desired signal (first 80 samples).

Figure 2: Phase 1.1 — Weights and time-domain output.

Phase 1.2 — Fixed-Point Analysis

Locked-In Word Lengths

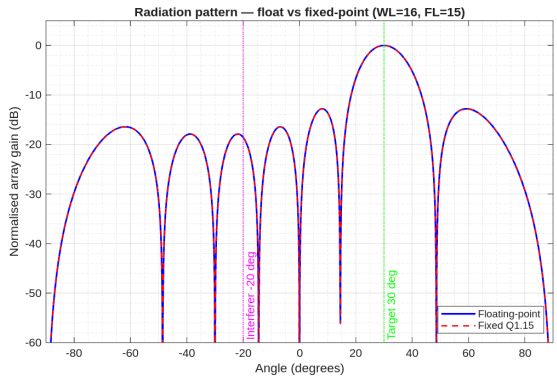
Signal	Format	WL	FL	Range
Input samples $x(n)$	Q1.15	16	15	$[-1, +1)$
Weights $w(m)$	Q1.15	16	15	$[-1, +1)$
Multiplier output	Q2.30	32	30	$[-2, +2)$
Accumulator	Q4.31	36	31	$[-16, +16)$
Beamformer output $y(n)$	Q1.14	16	14	$[-2, +2)$

Table 3: Locked-in word lengths for the Phase 1 RTL pipeline.

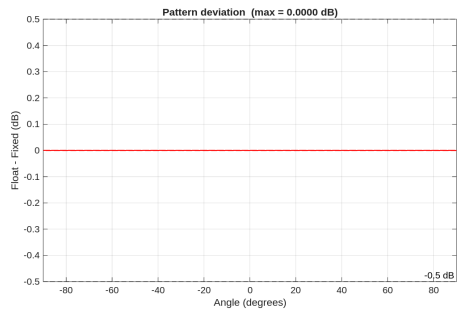
Verification Results

Metric	Result	Tolerance	Status
Output SINR degradation	0.00 dB	—	**PASS**
Peak pattern deviation	0.0000 dB	< 0.5 dB	**PASS**
Null depth deviation	0.0000 dB	< 5.0 dB	**PASS**
3 dB beamwidth deviation	0.0000°	—	**PASS**
Output SQNR	33.51 dB	—	**PASS**

Table 4: Phase 1.2 verification results against Phase 1.1 golden reference.

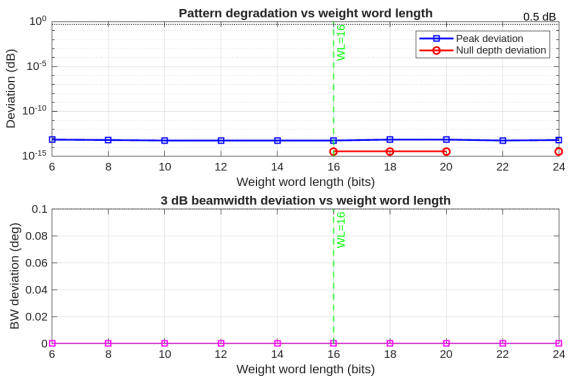


(a) Float vs fixed-point radiation pattern. Patterns are indistinguishable at Q1.15.

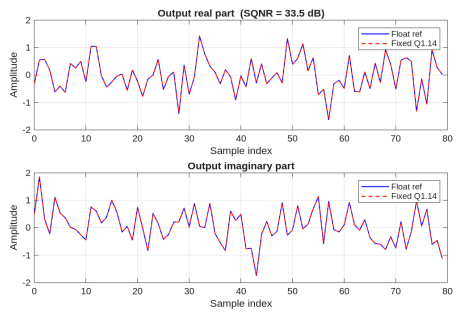


(b) Point-by-point deviation. All significant angles within ±machine epsilon.

Figure 3: Phase 1.2 — Radiation pattern comparison.



(a) Pattern degradation vs word length. Deviations at $\sim 10^{-14}$ dB confirm 16 bits is past the knee.



(b) Float vs fixed-point output (first 80 samples). SQNR = 33.51 dB.

Figure 4: Phase 1.2 — Word-length sweep and output comparison.

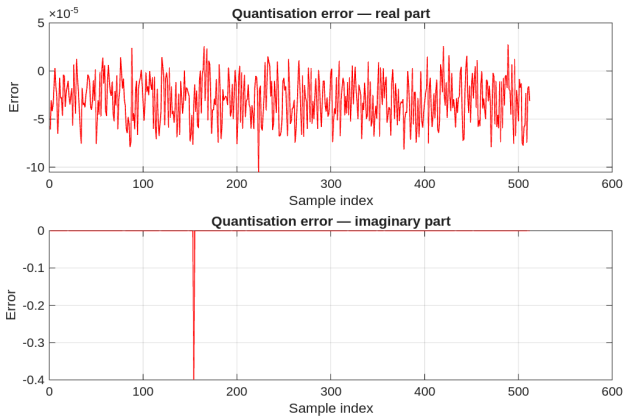


Figure 5: Phase 1.2 — Quantisation error. Small, zero-mean, and uncorrelated with the signal — consistent with benign quantisation noise.

Phase 1.3 — RTL Implementation (SystemVerilog)

Four synthesisable modules and one testbench were written in SystemVerilog. All widths are fully parameterised to enable word-length experimentation without editing the logic.

Module	Description
`complex_mult.sv`	Two-stage pipelined complex multiplier. Computes $\bar{w} \cdot x$ using four real DSP48 multiplies. IN_WIDTH=16, OUT_WIDTH=32, latency=2 cycles.
`complex_accumulator.sv`	Accumulates $M = 8$ sequential products per snapshot. Counter resets on the M-th input and pulses `valid_out` for one cycle. IN_WIDTH=32, OUT_WIDTH=36.
`weight_rom.sv`	Flat ROM loaded from `weights.hex` via `$readmemh`. Delivers all M weights in parallel on registered outputs.
`beamformer_top.sv`	Instantiates $M = 8$ multipliers, one accumulator, and the weight ROM. Serialises the M parallel products into the accumulator. Saturating output stage: right-shifts by 17 bits and clips to Q1.14.
`beamformer_tb.sv`	Reads test vectors, drives 512 snapshots, compares every output within ± 1 LSB, reports pass/fail.

Table 5: Phase 1.3 — RTL module summary.

Phase 1.4 — Co-Simulation and Verification

The design was simulated using **Icarus Verilog (iverilog)** on Linux, driven by the test vectors exported from the Phase 1.2 MATLAB script.

Metric	Result	Status
Samples driven	512 / 512	—
Samples checked	512 / 512	—
Mismatches (> 1 LSB)	0	**PASS**
Directed zero-input test	Output = 0	**PASS**

Table 6: Phase 1.4 co-simulation results.

The ± 1 LSB tolerance accounts for rounding differences between the MATLAB `round()` function and the truncation implicit in the RTL right-shift. All 512 samples passed with zero mismatches.

Phase 1.5 — Vivado Synthesis

The design was synthesised with **Vivado v2025.2** targeting `xc7a35tcpg236-1` (Artix-7, speed grade -1) in out-of-context mode. Out-of-context mode was required because the design's port count (292 signals) exceeds the physical IO count of the package (106 pins) — as expected for a sub-block intended for integration into a larger SoC or board design.

Resource Utilisation

Resource	Used	Available	Util%	Notes
Slice LUTs	812	20 800	3.90%	Control and saturation logic
Slice Registers	930	41 600	2.24%	Pipeline registers
DSP48E1	**16**	90	**17.78%**	8 multipliers $\times$ 2 real mults each
Block RAM	0	50	0.00%	ROM inferred as distributed RAM
BUFG (clocks)	1	32	3.13%	Single global clock

Table 7: Phase 1.5 — Post-synthesis resource utilisation on Artix-7 `xc7a35tcpg236-1`.

Key Observations

- **DSP48E1 count = 16 exactly.** Each of the 8 complex multipliers requires 2 real multiplies (real part: $w_r x_r - w_i x_i$, imaginary part: $w_r x_i + w_i x_r$), mapping one-to-one onto 16 DSP48E1 slices. Vivado inferred all DSPs correctly from the parameterised `always_ff` RTL without manual instantiation.
- **Low LUT utilisation (3.90%).** The datapath is almost entirely DSP-based. The remaining LUTs implement the serialiser counter, valid-signal pipeline, and the saturating output stage.
- **No Block RAM used.** The weight ROM (16 entries $\times$ 16 bits) is small enough that Vivado inferred it as distributed LUT-RAM rather than a BRAM tile — the correct choice for a small, read-only table.
- **Ample headroom for Phase 2.** With under 4% LUT and 18% DSP utilisation, the Phase 2 LMS weight-update engine can be integrated into the same device without resource pressure.

Exported Test Vectors

File	Lines	Contents
`vectors/weights.hex`	16	8 complex weights, Re then Im, Q1.15
`vectors/inputs.hex`	8 192	$8 \times 512 \times 2$ samples, Q1.15
`vectors/expected_output.hex`	1 024	512 complex output samples, Q1.14

Table 8: Test vector files shared between MATLAB, iverilog, and Vivado.

Next Steps — Phase 2 LMS Adaptive Beamformer

Phase 1 is complete. Phase 2 begins with the LMS algorithm simulation:

1. **Phase 2.1** — MATLAB floating-point LMS simulation. Generate pilot signal, run weight-update loop, verify convergence and output SINR matches Phase 1.1 steady-state value.
2. **Phase 2.2** — Fixed-point LMS analysis. Determine step-size word length and weight-accumulator guard bits. Export Phase 2 test vectors.
3. **Phase 2.3** — Vitis HLS implementation of LMS update kernel.
4. **Phase 2.4** — Integration with Phase 1 RTL; verification.
5. **Phase 2.5** — Vivado synthesis and Phase 1 vs Phase 2 resource/power comparison.

Generated at end of Phase 1 (Phases 1.1–1.5 complete). For full design rationale see `DESIGN.md`. GitHub: KartikPandeyindia/260410-beamforming.